

Programmazione 1

Corso c

15/10/2023
> Array e Matrici

Alessandro Mazzei

alessandro.mazzei@unito.it

Dipartimento di Informatica
Università degli Studi di Torino



UNIVERSITÀ
DI TORINO

Outline

- Funzione Chiamante e Funzione Chiamata
- Esercizi Array
- Ancora binary search: intervalli semi-aperti
- Matrici, Memoria e funzioni
- Matrici sfrangiate
- Esercizi Matrici
- Malloc e Heap

Outline

- Funzione Chiamante e Funzione Chiamata
- Esercizi Array
- Ancora binary search: intervalli semi-aperti
- Matrici, Memoria e funzioni
- Matrici sfrangiate
- Esercizi Matrici
- Malloc e Heap

Funzione Chiamante e Funzione Chiamata

Funzione che deve calcolare un **output scalare** con **input scalari** (es. `isOdd`).

Due possibilità:

1. La funzione chiamante passa l'input alla chiamata per **valore attraverso i parametri formali** e la chiamata passa l'output alla chiamante attraverso il **return**
2. La funzione chiamante passa l'input alla chiamata **per valore attraverso i parametri formali** e la chiamata passa l'output alla chiamante per **riferimento *indiretto/simulato* attraverso un parametro formale di tipo puntatore**

ATTENZIONE: nel caso 2 l'output deve essere per forza un puntatore!!!

Funzione Chiamante e Funzione Chiamata

- Perché l'input non viene passato per riferimento indiretto/simulato?

Golden Rule:

The usual rule is to pass quantities by value unless the program needs to alter the value, in which case you pass a pointer (CPrimerPlus)

Funzione Chiamante e Funzione Chiamata

Funzione che deve calcolare un **output vettoriale** con **input vettoriale** (es. `onlyOdd`).

Una sola possibilità:

1. La funzione chiamante passa il vettore in input alla chiamata per riferimento *simulato* attraverso 2 parametri formali (1p e `1int/size_t`) e la chiamata passa l'output alla chiamante per riferimento *simulato* attraverso 2 parametri formali (1p e `1int/size_t`)

Dove sono allocati i vettori? Perché?

Funzione Chiamante e Funzione Chiamata

- Un INPUT (vettoriale) può coincidere con l'OUTPUT (vettoriale)?
- Esempio:
 - Cubo
 - `scanf` (scalare)
- Non voglio permettere di far variare il parametro: `const`

Outline

- Funzione Chiamante e Funzione Chiamata
- Esercizi Array
- Ancora binary search: intervalli semi-aperti
- Matrici, Memoria e funzioni
- Matrici sfrangiate
- Esercizi Matrici
- Malloc e Heap

Filtri 1: IN vettore, OUT vettore

- `void filtroDispariIt(int a[], size_t aLen, int b[], size_t* bLen)` modifica b, memorizzandovi tutti e soli gli elementi di a il cui valore è dispari;
- `void filtroPosizioniPariIt(int a[], size_t aLen, int b[], size_t* bLen)` modifica b, memorizzandovi tutti e soli gli elementi di a che occupano posizioni pari;
- `void filtroPosizioniDispariElementiPariIt(int a[], size_t aLen, int b[], size_t* bLen)` modifica b, memorizzandovi tutti e soli gli elementi di a che occupano una posizione dispari avendo, simultaneamente, valore pari;

Attenzione alla signature!!!

- `void filtroDispariIt(int a[], size_t aLen, int b[], size_t* bLen);`
- `void filtroDispariIt(int a[], size_t aLen, int b[], size_t* bLen, size_t LMAX);`
- `int filtroDispariIt(int a[], size_t aLen, int b[], size_t LMAX);`

Filtri 2: IN vettore, OUT vettore

- `void filtroIntervalloIt (int a[], size_t aLen, int b[], size_t* bLen, int min, int max)` **modifica b**, memorizzandovi tutti e soli gli elementi di a il cui valore è compreso nell'intervallo [min, max], in cui gli estremi sono inclusi;
- `void filtroIndiciDoppioRiferimento (int a[], size_t aLen, int b[], size_t* bLen, int riferimento)` **modifica b**, memorizzandovi tutti e soli gli indici degli elementi di a che hanno valore doppio di riferimento.

multipliTre: IN vettori, OUT vettore

Scrivere una funzione C iterativa di nome `multipliTre` con le seguenti caratteristiche:

- `multipliTre` dipende dai seguenti parametri formali:
 - due array `a` e `b` di tipo intero, con dimensione massima `L_MAX > 0`, visti come input e tali che la parte effettivamente inizializzata sia delimitata dai valori in `aLen` e `bLen`, e un array `r` di tipo intero, con dimensione massima `L_MAX > 0`, visto come output;
- `multipliTre` restituisce `r` modificata come segue:
 - Per ogni indice `i` nell'intervallo `[0,minimo{aLen, bLen}]`, in cui l'estremo inferiore è incluso e quello superiore escluso, l'elemento `r[i]` assume il valore `true` se almeno uno tra gli elementi di posizione `i` in `a` e `b` è multiplo di 3.
- `multipliTre` modifica anche il valore di `rLen` coerentemente con l'idea che un array, in questo caso `r`, è completamente descritto quando è noto il numero di elementi effettivamente utilizzati in esso.

IN array, OUT scalare

- Scrivere una funzione `C int sommaTuttiElementi(int a[], int la)` che, applicata ad un array `a`, restituisce la somma di tutti gli elementi in `a`.
- Scrivere una funzione `C int moltiplicaTuttiElementi(int a[], int la)` che, applicata ad un array `a`, restituisce la moltiplicazione di tutti gli elementi in `a`.
- Scrivere una funzione `C float sommaFrazioniTuttiElementi(int a[], int la)` che, applicata ad un array `a`, restituisce la somma di tutte le frazioni che si ottengono dividendo ogni elemento in `a` col successivo, ammesso che quest'ultimo esista.

CodeRunner

- Lab05-Es1 Somma selettiva
- Lab06-Es1 Statistiche sequenze

Outline

- Funzione Chiamante e Funzione Chiamata
- Esercizi Array
- Ancora binary search: intervalli semi-aperti
- Matrici, Memoria e funzioni
- Matrici sfrangiate
- Esercizi Matrici
- Malloc e Heap

Intervalli chiusi e semiaperti

- BinarySearch alla Deitel: $[0, n-1]$ vs. $[0, n)$
 - Left e Right potevano diventare negativi (sono size_t!!!)
- Possiamo riscrivere la binary search con un intervallo semiaperto? Quali sono i vantaggi?
- $[L, R)$
 - Il numero di elementi dell'intervallo è: $R-L$
 - L'intervallo è vuoto (non ha elementi) se: $R == L$
 - Se l'intervallo non è vuoto, allora:
 - L è il primo elemento
 - $R-1$ è l'ultimo elemento

BinarySearch3

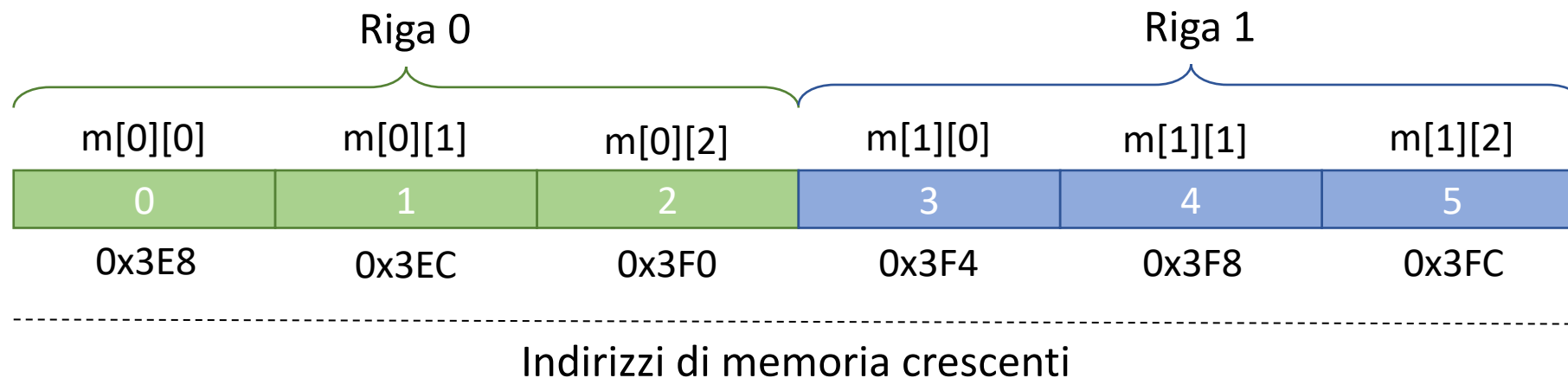
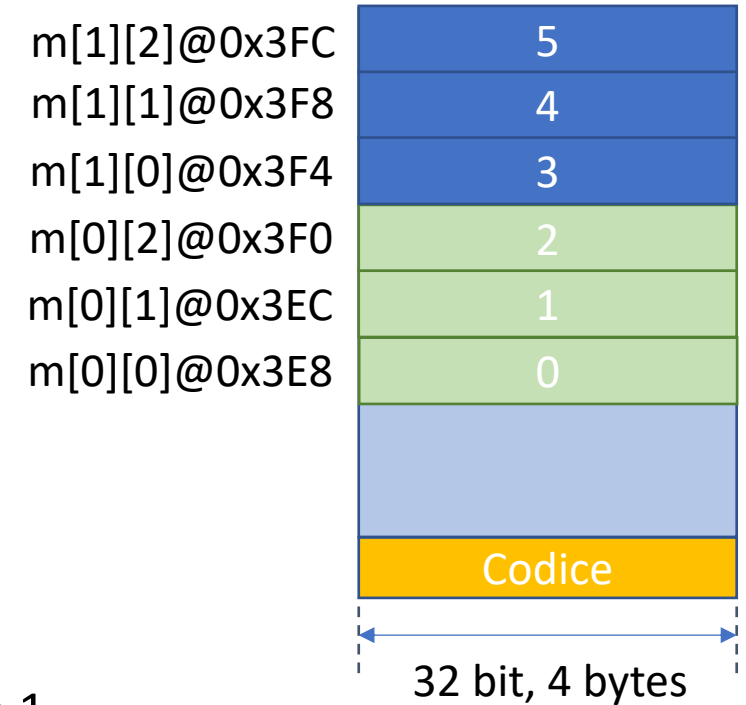
```
int binarySearch3(const int a[], int key, size_t left, size_t right) {
    int ret = -1;
    // loop until the interval [l, r) is not empty AND the key has not been found
    while (right - left > 0 && ret == -1) {
        size_t middle = (left + right) / 2; // determine middle subscript
        // display subarray used in this loop iteration
        printRow(a, left, middle, right);
        // if key matches, return middle subscript
        if (key == a[middle]) {
            ret = middle;
        }
        else if (key < a[middle]) { // if key < b[middle], adjust right
            right = middle; // next iteration searches left end of array
        }
        else { // key > b[middle], so adjust left
            left = middle+1; // next iteration searches right end of array
        }
    }
    // end while
    return ret;
}
```

Outline

- Funzione Chiamante e Funzione Chiamata
- Esercizi Array
- Ancora binary search: intervalli semi-aperti
- Matrici, Memoria e funzioni
- Matrici sfrangiate
- Esercizi Matrici
- Malloc e Heap

Modello di memoria

- Matrice m è composta da M_ROWS * M_COLS celle di memoria allocate contiguamente nello stack in *row major order*

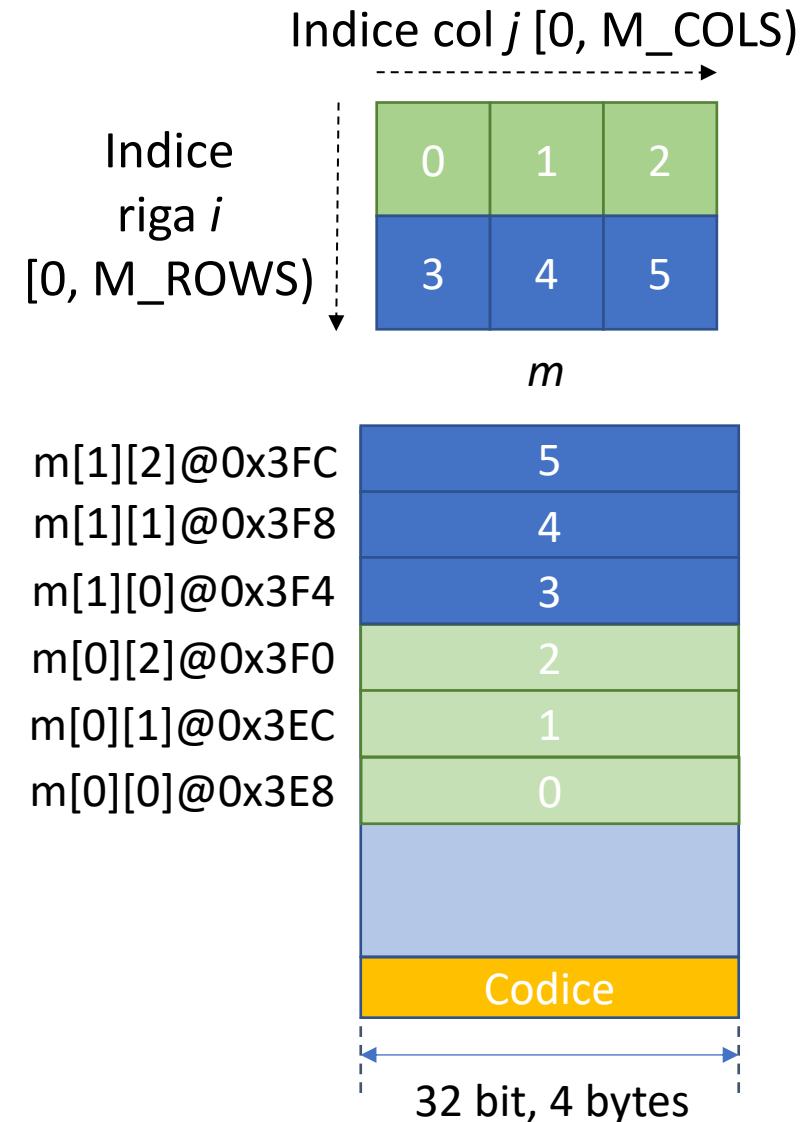


Modello di memoria

```
#define M_ROWS 2
#define M_COLS 3

int main(void) {
    int m[M_ROWS][M_COLS] = {{0,1,2}, {3,4,5}};
    for (int i = 0; i < M_ROWS; i++) {
        for (int j = 0; j < M_COLS; j++) {
            printf ("m[%d][%d]@%p vale %d\n", i, j,
&m[i][j], m[i][j]);
        }
    }
}
```

```
m[0][0]@0x3E8 vale 0
m[0][1]@0x3EC vale 1
m[0][2]@0x3F0 vale 2
m[1][0]@0x3F4 vale 3
m[1][1]@0x3F8 vale 4
m[1][2]@0x3FC vale 5
```



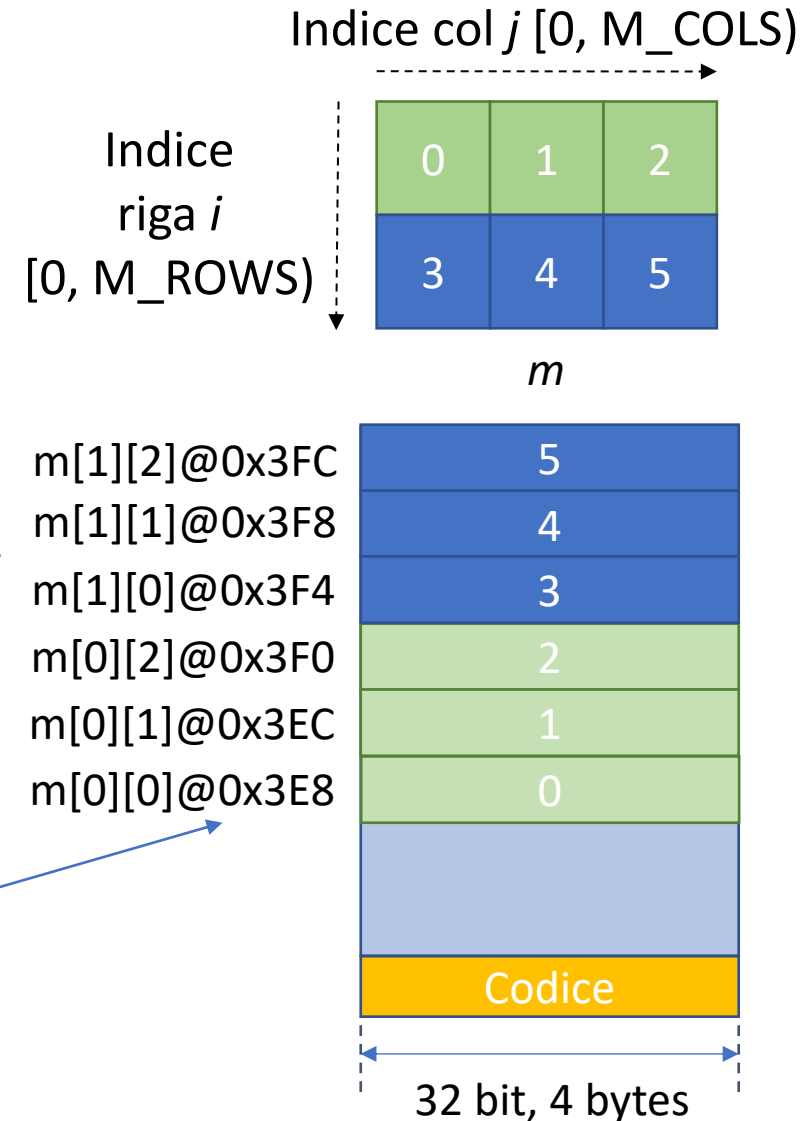
Puntatori a matrici

- Il nome della matrice è un puntatore alla cella di indirizzo più basso, ovvero `m[0][0]`

```
printf("m[0][0]@%p vale %d", m, *(int*)m);
```

`m[0][0] @ 0x3E8`

m è l'indirizzo di questa parola in memoria



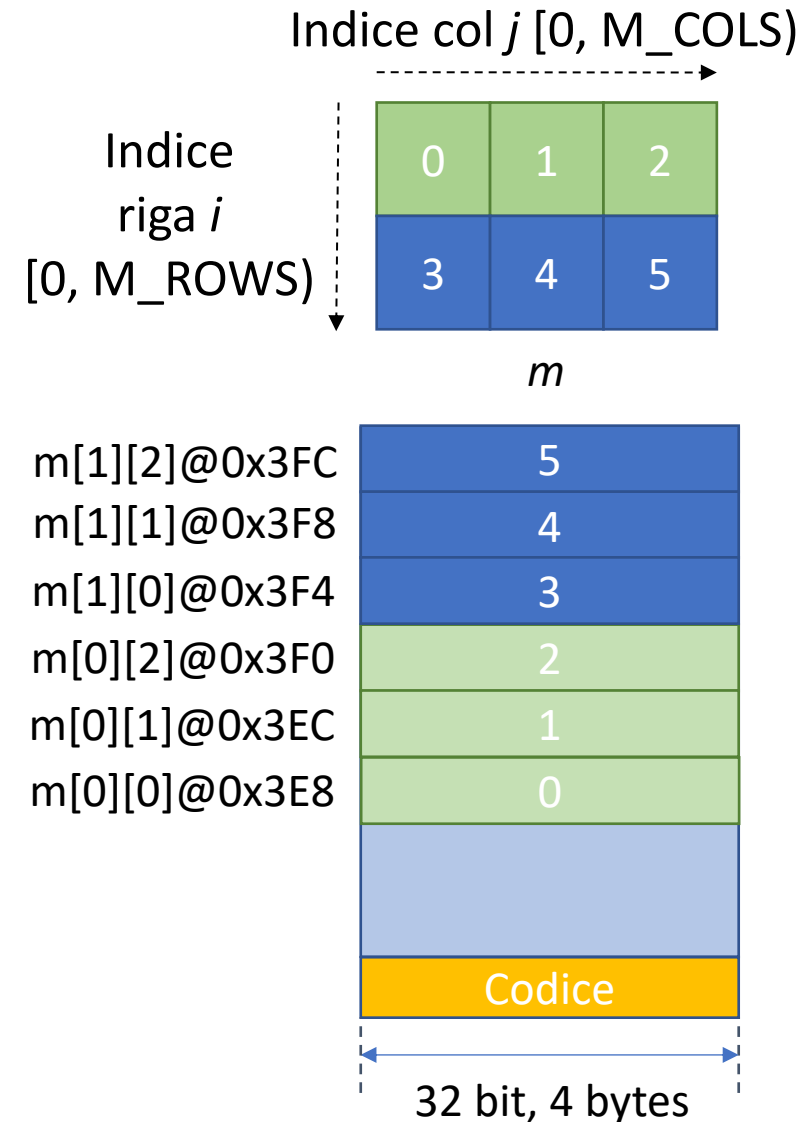
Puntatori a matrici

- Il nome della matrice è un puntatore alla cella di indirizzo più basso, ovvero `m[0][0]`

```
printf("m[0][0]@%p vale %d", m, * (int*)m);
```

`m[0][0] @ 0x3E8 vale 0`

Casting a `int*` di `m`
prima della
dereferenziazione



Puntatori a matrici

- Il nome della matrice è un puntatore alla cella di indirizzo più basso, ovvero `m[0][0]`

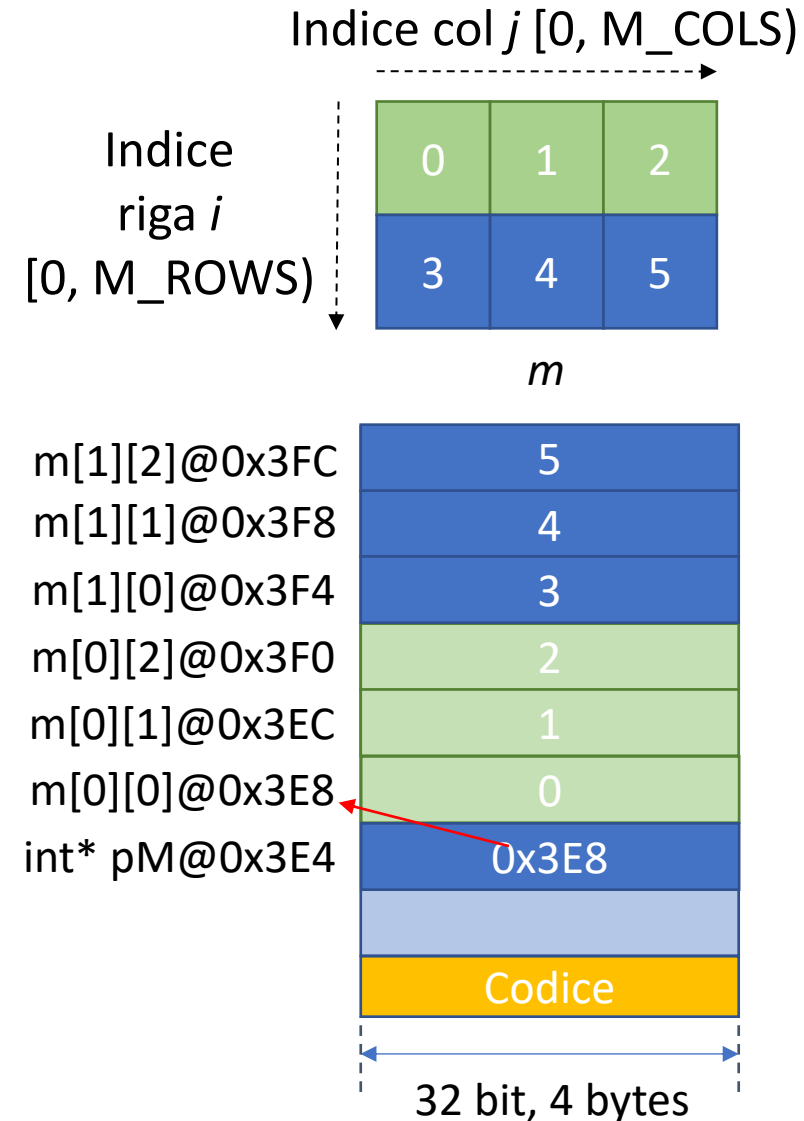
```
printf("m[0][0]@%p vale %d", m, *(int*)m);
```

`m[0][0] @ 0x3E8 vale 0`

- Si può ciclare sugli elementi di `m` dato un puntatore di lavoro (e opportuno casting)

```
int* pM = (int*) m;
```

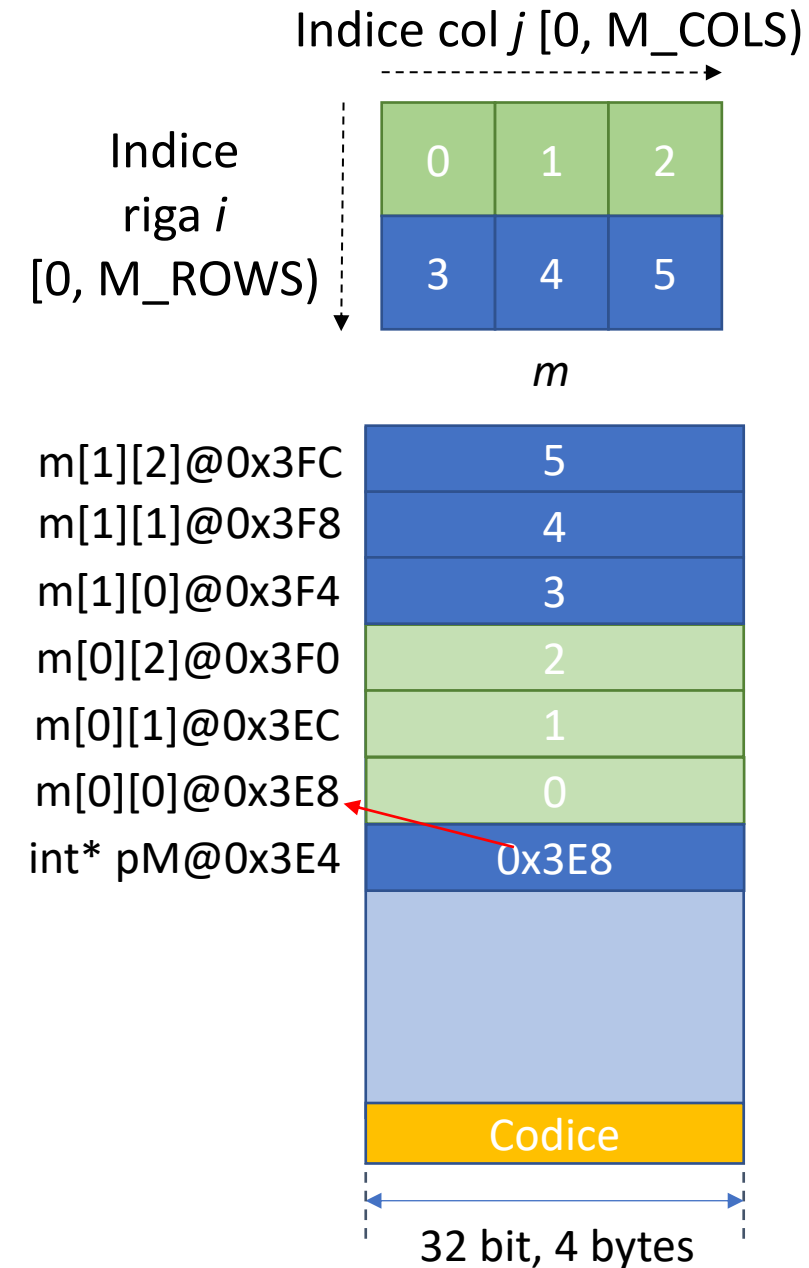
Casting
da `int*[2]`
a `int*`



Puntatori a matrici

```
#define M_ROWS 2
#define M_COLS 3

int main(void) {
    int m[M_ROWS][M_COLS] = {{0,1,2},
    → {3,4,5}};
    int* pM = (int*) m;
    for (int i = 0; i < M_ROWS; i++) {
        for (int j = 0; j < M_COLS; j++) {
            printf ("m[%d][%d]@%p vale %d\n", i,
                j, pM, *pM);
            pM++;
        }
    }
}
```

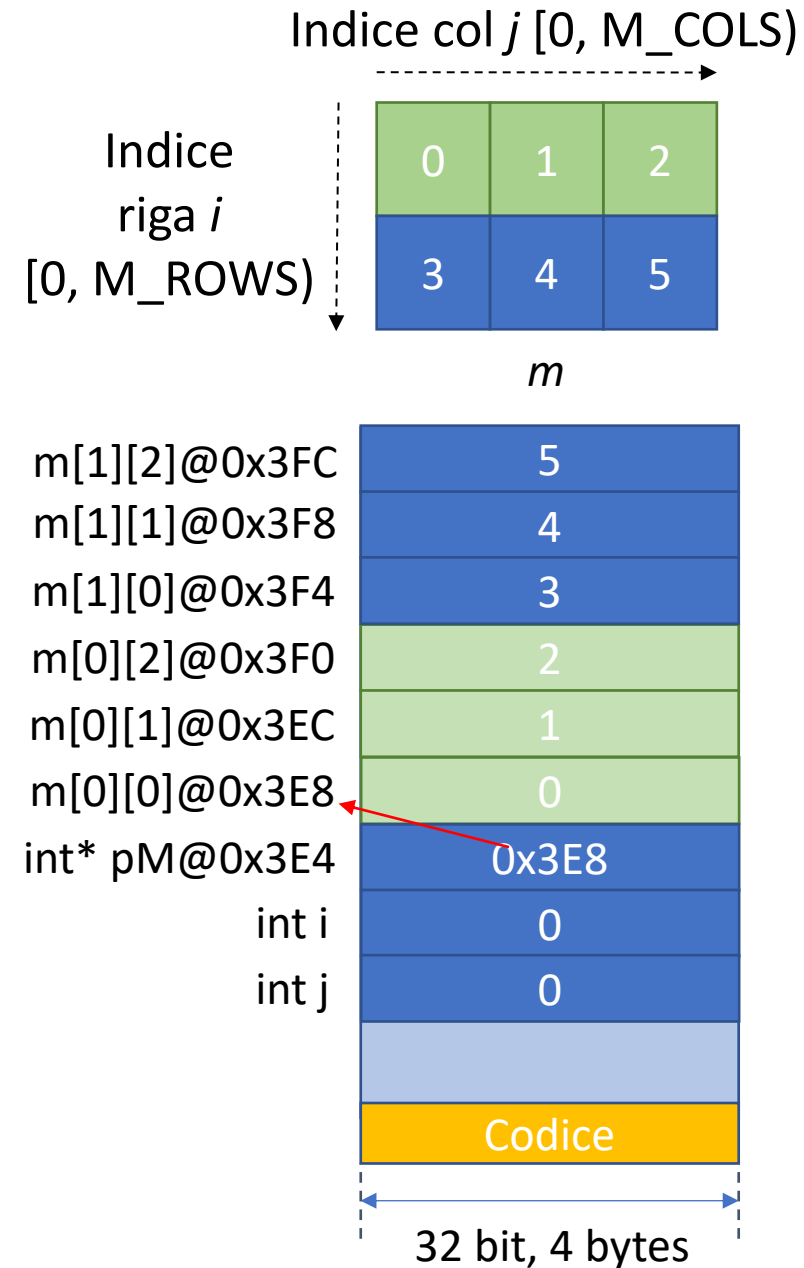


Puntatori a matrici

```
#define M_ROWS 2
#define M_COLS 3

int main(void) {
    int m[M_ROWS][M_COLS] = {{0,1,2},
    {3,4,5}};
    int* pM = (int*) m;
    for (int i = 0; i < M_ROWS; i++) {
        for (int j = 0; j < M_COLS; j++) {
            printf ("m[%d][%d]@%p vale %d\n", i,
            j, pM, *pM);
            pM++;
        }
    }
}
```

m[0][0]@0x3E8 vale 0

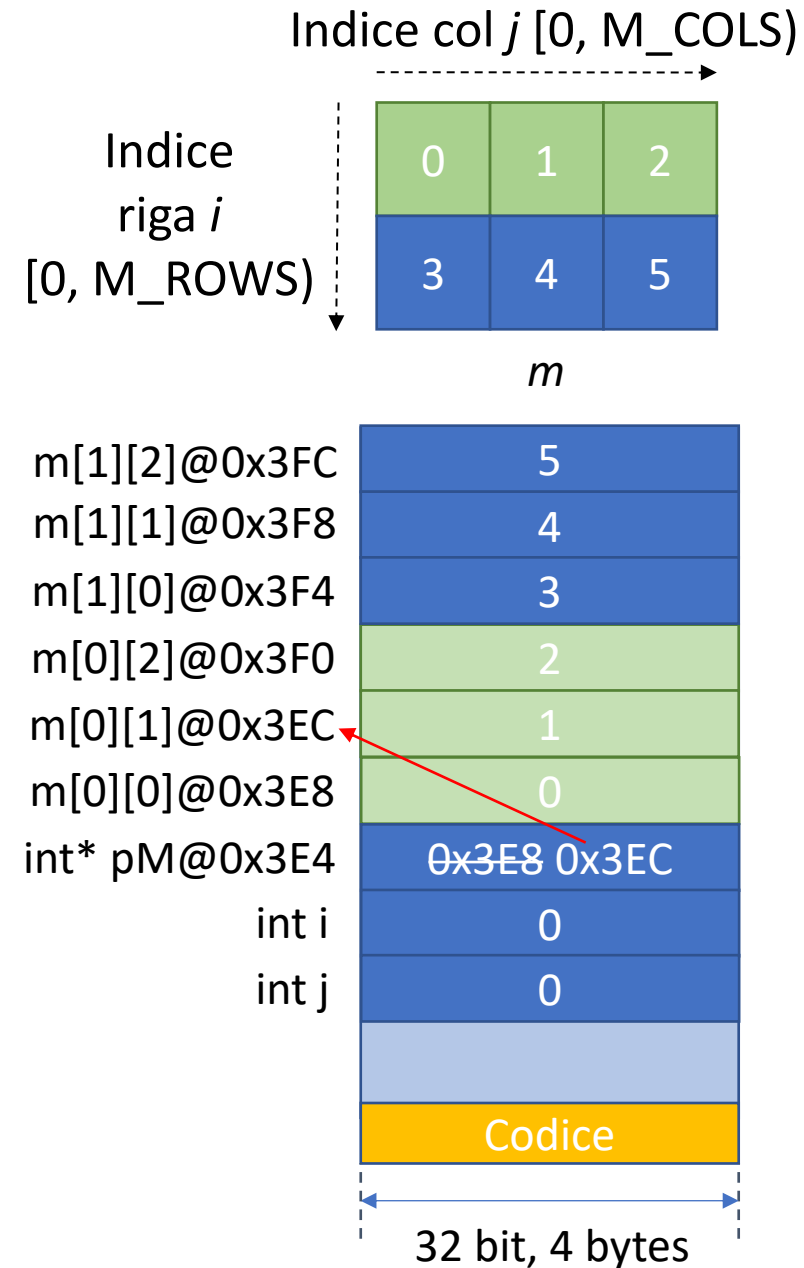


Puntatori a matrici

```
#define M_ROWS 2
#define M_COLS 3

int main(void) {
    int m[M_ROWS][M_COLS] = {{0,1,2},
    {3,4,5}};
    int* pM = (int*) m;
    for (int i = 0; i < M_ROWS; i++) {
        for (int j = 0; j < M_COLS; j++) {
            printf ("m[%d][%d]@%p vale %d\n", i,
j, pM, *pM);
            pM++;
        }
    }
}
```

m[0][0]@0x3E8 vale 0

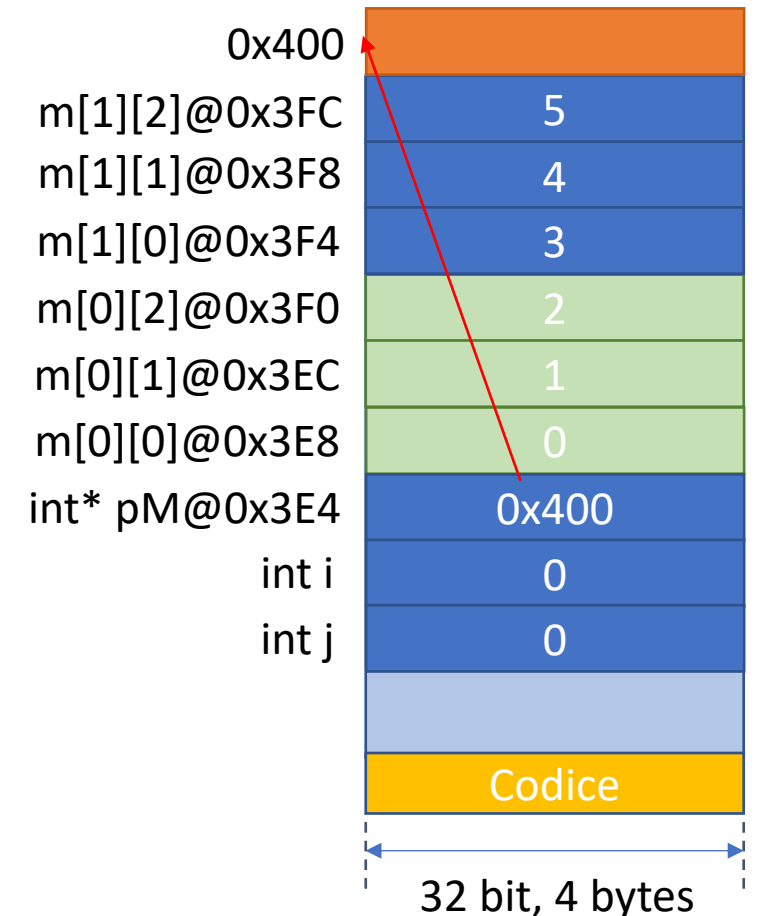


Puntatori a matrici

```
#define M_ROWS 2
#define M_COLS 3

int main(void) {
    int m[M_ROWS][M_COLS] = {{0,1,2},
    {3,4,5}};
    int* pM = (int*) m;
    for (int i = 0; i < M_ROWS; i++) {
        for (int j = 0; j < M_COLS; j++) {
            printf ("m[%d][%d]@%p vale %d\n", i,
j, pM, *pM);
            pM++;
        } //end for su j
    } // end for su i
}
```

```
m[0][0]@0x3E8 vale 0
m[0][1]@0x3EC vale 1
m[0][2]@0x3F0 vale 2
m[1][0]@0x3F4 vale 3
m[1][1]@0x3F8 vale 4
m[1][2]@0x3FC vale 5
```

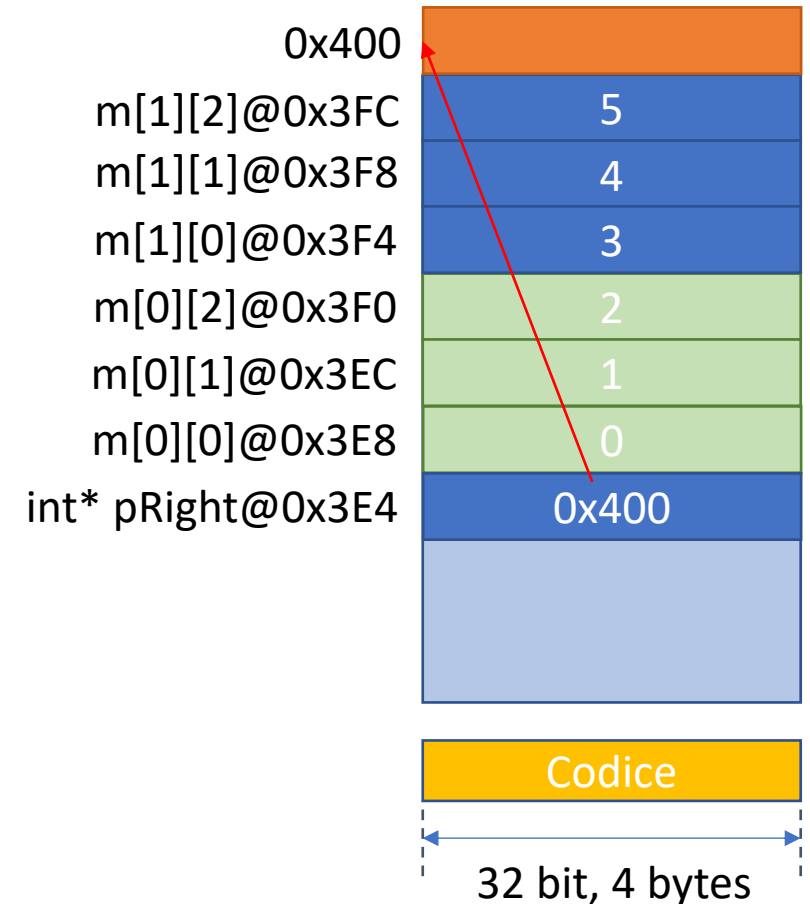


Puntatori a matrici

```
#define M_ROWS 2
#define M_COLS 3

int main(void) {
    int m[M_ROWS][M_COLS] = {{0,1,2},
    {3,4,5}};
    ➔ int* pRight = (int*) m + (M_ROWS *  
M_COLS) * sizeof(int);
    for (int* pM = (int*) m; pM < pRight;  
pM++) {
        printf ("int@%p vale %d\n", pM, *pM);
    }
}
```

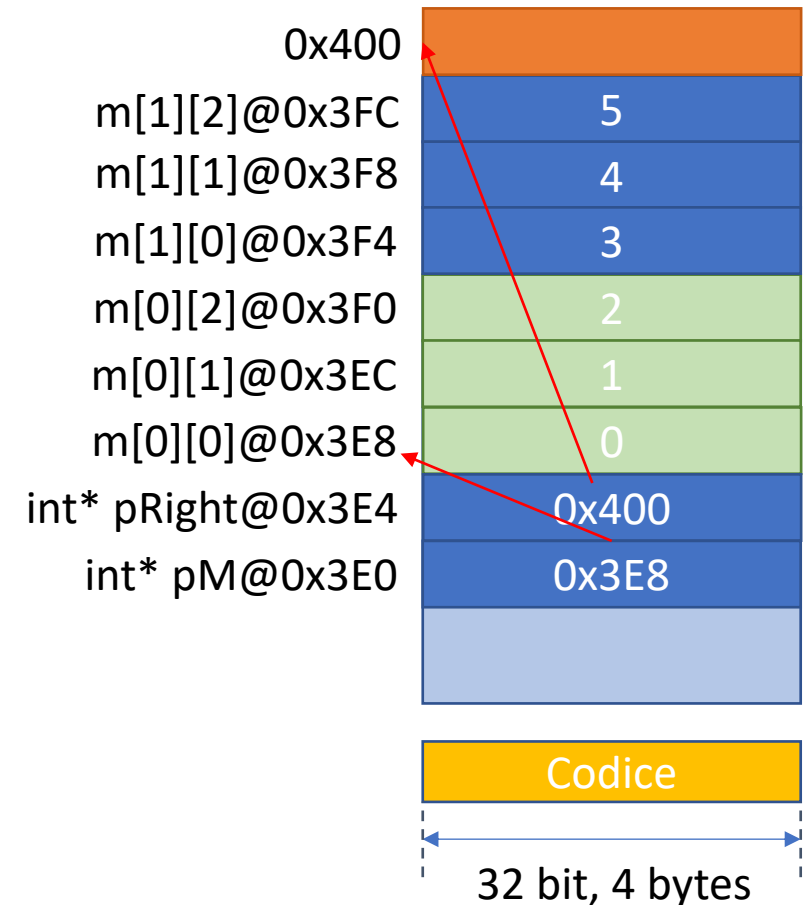
Vale l'aritmetica
dei puntatori!



Puntatori a matrici

```
#define M_ROWS 2
#define M_COLS 3

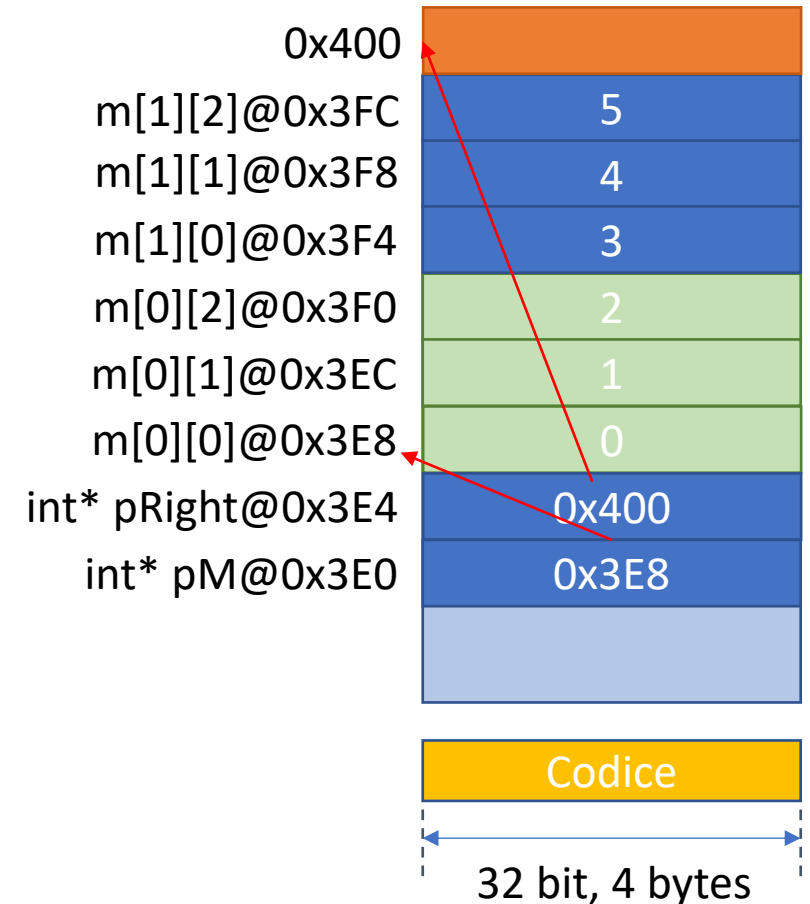
int main(void) {
    int m[M_ROWS][M_COLS] = {{0,1,2},
    {3,4,5}};
    int* pRight = (int*) m + (M_ROWS *
M_COLS);
    ➔ for (int* pM = (int*) m; pM < pRight;
pM++) {
        printf ("int@%p vale %d\n", pM, *pM);
    }
}
```



Puntatori a matrici

```
#define M_ROWS 2
#define M_COLS 3

int main(void) {
    int m[M_ROWS][M_COLS] = {{0,1,2},
    {3,4,5}};
    int* pRight = (int*) m + (M_ROWS *
M_COLS);
    ➔ for (int* pM = (int*) m; pM < pRight;
pM++) {
        printf ("int@%p vale %d\n", pM, *pM);
    }
}
```

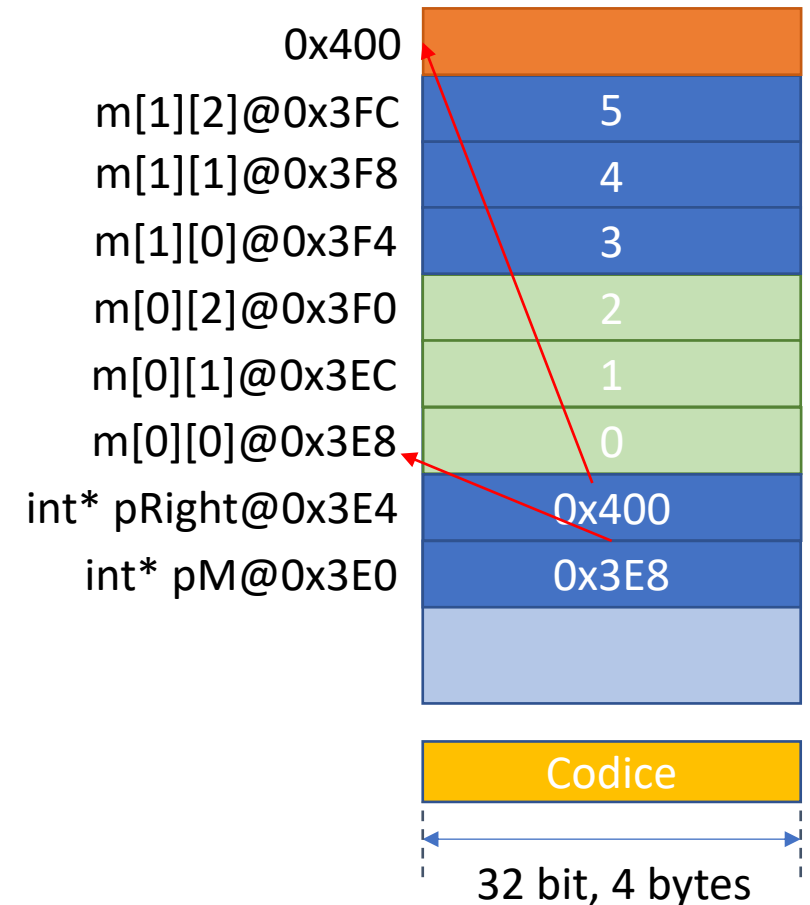


Puntatori a matrici

```
#define M_ROWS 2
#define M_COLS 3

int main(void) {
    int m[M_ROWS][M_COLS] = {{0,1,2},
    {3,4,5}};
    int* pRight = (int*) m + (M_ROWS *
M_COLS);
    for (int* pM = (int*) m; pM < pRight;
pM++) {
        printf ("int@%p vale %d\n", pM, *pM);
    }
}
```

m[0][0]@0x3E8 vale 0

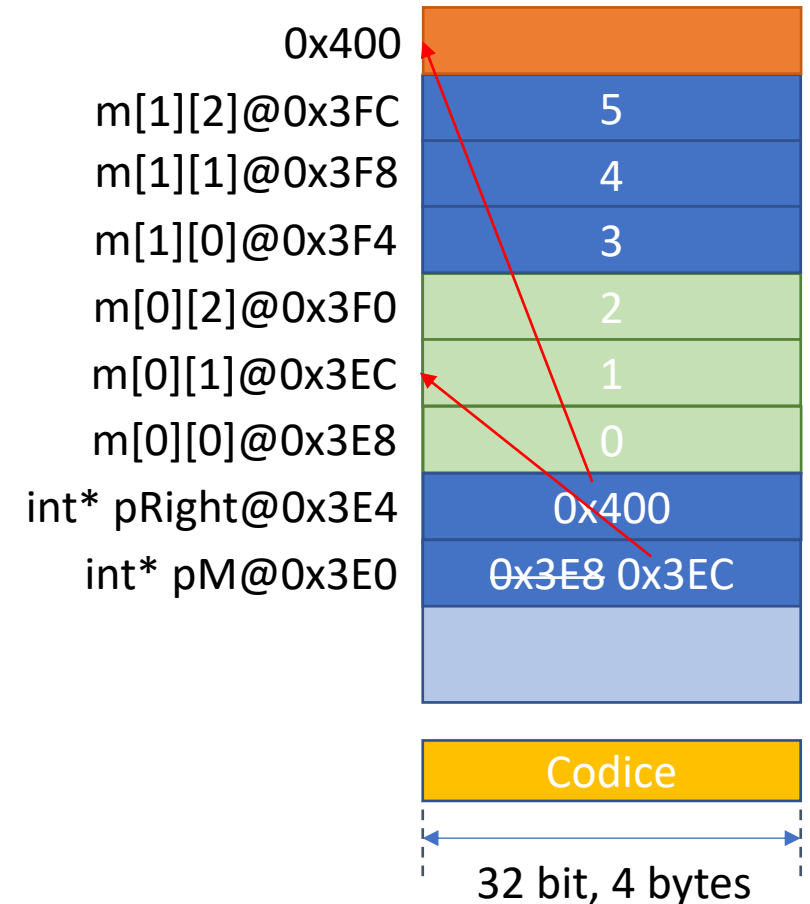


Puntatori a matrici

```
#define M_ROWS 2
#define M_COLS 3

int main(void) {
    int m[M_ROWS][M_COLS] = {{0,1,2},
    {3,4,5}};
    int* pRight = (int*) m + (M_ROWS *
M_COLS);
    for (int* pM = (int*) m; pM < pRight;
➔ pM++) {
        printf ("int@%p vale %d\n", pM, *pM);
    }
}
```

m[0][0]@0x3E8 vale 0

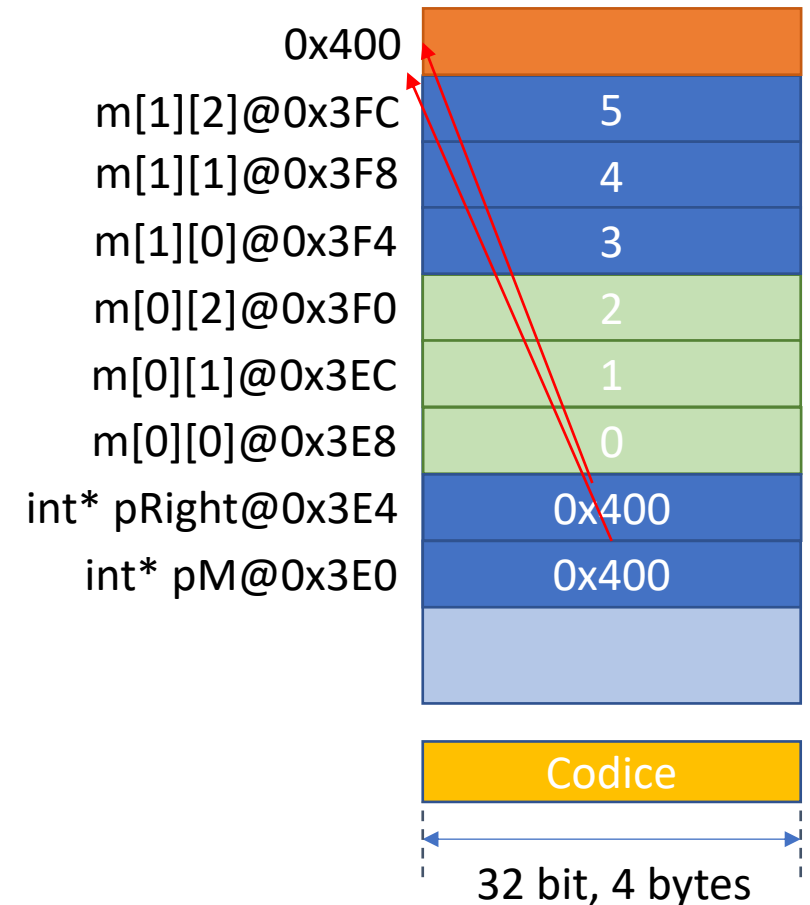


Puntatori a matrici

```
#define M_ROWS 2
#define M_COLS 3

int main(void) {
    int m[M_ROWS][M_COLS] = {{0,1,2},
    {3,4,5}};
    int* pRight = (int*) m + (M_ROWS *
M_COLS);
    for (int* pM = (int*) m; pM < pRight;
pM++) {
        printf ("int@%p vale %d\n", pM, *pM);
    }
}
```

m[0][0]@0x3E8 vale 0
m[0][1]@0x3EC vale 1
m[0][2]@0x3F0 vale 2
m[1][0]@0x3F4 vale 3
m[1][1]@0x3F8 vale 4
m[1][2]@0x3FC vale 5

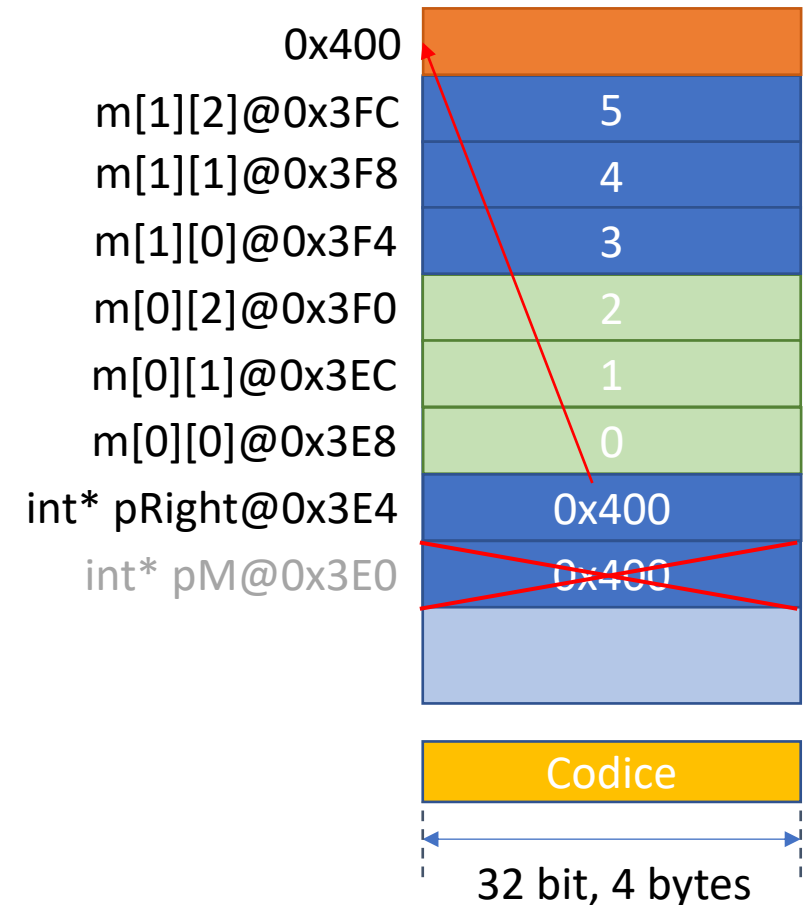


Puntatori a matrici

```
#define M_ROWS 2
#define M_COLS 3

int main(void) {
    int m[M_ROWS][M_COLS] = {{0,1,2},
    {3,4,5}};
    int* pRight = (int*) m + (M_ROWS *
M_COLS);
    for (int* pM = (int*) m; pM < pRight;
pM++) {
        printf ("int@%p vale %d\n", pM, *pM);
    }
} //qui
```

m[0][0]@0x3E8 vale 0
m[0][1]@0x3EC vale 1
m[0][2]@0x3F0 vale 2
m[1][0]@0x3F4 vale 3
m[1][1]@0x3F8 vale 4
m[1][2]@0x3FC vale 5



Chiamata a funzione con matrici (ancora!?)

Signature *classica*: ho una costante per le colonne

```
#define STUDENTS 3
#define EXAMS 4

int minimum(const int grades[][EXAMS], size_t pupils, size_t tests);
int maximum(const int grades[][EXAMS], size_t pupils, size_t tests);
double average(const int setOfGrades[], size_t tests);
void printArray(const int grades[][EXAMS], size_t pupils, size_t tests);
```

Chiamata a funzione con matrici (ancora!?)

Signature *moderna*: ho delle variabili per le colonne (e le righe)

```
bool OgniRigaEsisteColonnaPari(int r, int c, int m[r][c]);
```

In questo caso sto implicitamente definendo un VLA: funziona solo su compilatori *moderni* (C99)

Funziona sia:

- su array-argomenti VLA
- su array-argomenti costanti